Mobile Application Development Laboratory

Submitted by

SUDHARSAN S 231901054

in partial fulfillment of the award of the degree
of

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI
An Autonomous Institute
CHENNAI

BONAFIDE CERTIFICATE

Certified that this project **"JAZZ MUSIC PLAYER"** is the bonafide work of **"SUDHARSAN S"** who carried out the project work under my supervision.

SIGNATURE

Mrs. Seethalakshmi
Assistant Professor,
Dept. of Computer Science and Engineering,
(CYBER SECURITY),
Rajalakshmi Engineering College,
(Autonomous),
Thandalam, Chennai - 602 105.

This project report is submitted for the viva voce examination to be held on _____

EXAMINER

ACKNOWLEDGEMENT

I express my sincere thanks to our beloved and honourable chairman Mr. Meganathan S and the chairperson Dr. Thangam Meganathan M for their timely support and encouragement.

I express my sincere thanks to our Head of the Department Mr. Benedict JN, for encouragement and being ever supporting force during my project work.

I also extend my sincere and hearty thanks to our internal guide Mrs. Seethalakshmi, for her valuable guidance and motivation during the completion of this project.

SUDHARSAN S 231901054

TABLE OF CONTENTS

CHAPTER 1
INTRODUCTION

The modern mobile application landscape is increasingly shifting toward cross-platform efficiency and high-performance rendering. This project presents Jazz Music Player, a specialised mobile application designed to bridge the gap between native performance and rapid multi-platform development for music enthusiasts and audiophiles. While generic music players often provide a "one-size-fits-all" approach, Jazz Music Player addresses the specific needs of users by delivering highly personalised audio playback, library management, and music discovery capabilities. Utilising the Google-developed Dart language and Flutter framework, the application delivers a robust and fluid user experience across Android devices.

At its core, Jazz Music Player solves the problem of fragmented audio tooling by intercepting and managing all aspects of local music playback via a background audio service. The architecture integrates a robust offline-first data strategy using the sqflite relational database library. This ensures that the user's music library, playlists, and listening history remain persistent and secure locally on the device. Furthermore, the app supports hardware integration such as lockscreen media controls and notification-panel playback widgets via Android's MediaSession API.

1.1. Key Features

- High-Performance UI: Utilises the Material Design 3 framework for consistent rendering across Android devices.
- Offline First Capability: Robust local storage through the sqflite database library, ensuring data persistence without an internet connection.
- Multi-Format Playback: Native support for MP3, FLAC, WAV, and AAC with gapless playback and configurable crossfade transitions.
- Equaliser with Presets: 10-band equaliser featuring presets — Bass Boost, Vocal Clarity, Classical, Rock, Jazz, and Electronic.
- State Restoration: Lifecycle-aware architecture via Riverpod that allows the application to recover its exact UI state after system-initiated termination.
- Video-to-Audio Conversion: FFmpeg-powered in-app conversion with format and bitrate selection plus real-time progress reporting
- Smart Playlists: Automatically generated Recently Played and Most Played playlists driven by listening-history data.

CHAPTER 2
SCOPE OF THE PROJECT

The foundation of the app is establishing a "Dynamic Music Profile" that adapts based on the user's current listening context and preferences.

- Format-Specific Parameters: Selection of primary audio codecs (MP3, FLAC, WAV, AAC) to adjust decoding pipeline and audio quality settings.
- Background Playback: Real-time audio session management via the audio_service package to maintain playback when the app is minimised or the device is locked.
- Dynamic Library Partitioning: Classification of tracks into songs, albums, artists, and folders based on MediaStore metadata and file-system structure.
- Intelligent Scheduling: Background scans and metadata extraction scheduled based on device idle state and battery level to avoid interrupting playback.
- Real-Time Data Monitoring: An algorithm that calculates storage usage and estimates remaining capacity for new downloads and conversions.
- Progression Tracking: Logging of play count, last-played timestamp, and per-album completion percentages to power smart playlists.
- Video-to-Audio Conversion Pipeline: FFmpeg-driven extraction with batch support, format selection (MP3, AAC, FLAC), and bitrate options (128 kbps, 256 kbps, 320 kbps).
- Permission Management: Graceful runtime permission flow targeting Android 13+ READ_MEDIA_AUDIO and FOREGROUND_SERVICE_MEDIA_PLAYBACK.

CHAPTER 3
ARCHITECTURE

The application follows a **Clean Architecture modular design** for scalability and maintainability.

Presentation Layer

Built using Flutter Widgets (StatefulWidget and ConsumerWidget via Riverpod) and specialised rendering classes like BottomNavigationBar, PageView, and CustomPainter for the audio visualiser. Screens: Home, Library, Now Playing, Playlists, Search, Converter, and Settings.

Business Logic Layer

Handles state transitions and data processing via Dart asynchronous operations (async/await, Streams) and Riverpod AsyncNotifiers. Use-case classes encapsulate all domain rules — GetTracksUseCase, PlayTrackUseCase, ConvertVideoUseCase.

Data Access Layer

Uses sqflite (SQLite for Flutter) with a DatabaseHelper singleton to interact with the device's local storage. Manages all application data: tracks, playlists, listening history, conversion jobs, and cached metadata.

Hardware Abstraction Layer

Facilitated by audio_service and just_audio for media session management, on_audio_query for MediaStore access, and ffmpeg_kit_flutter for video-to-audio conversion executed in background Dart isolates.

Architecture Diagram

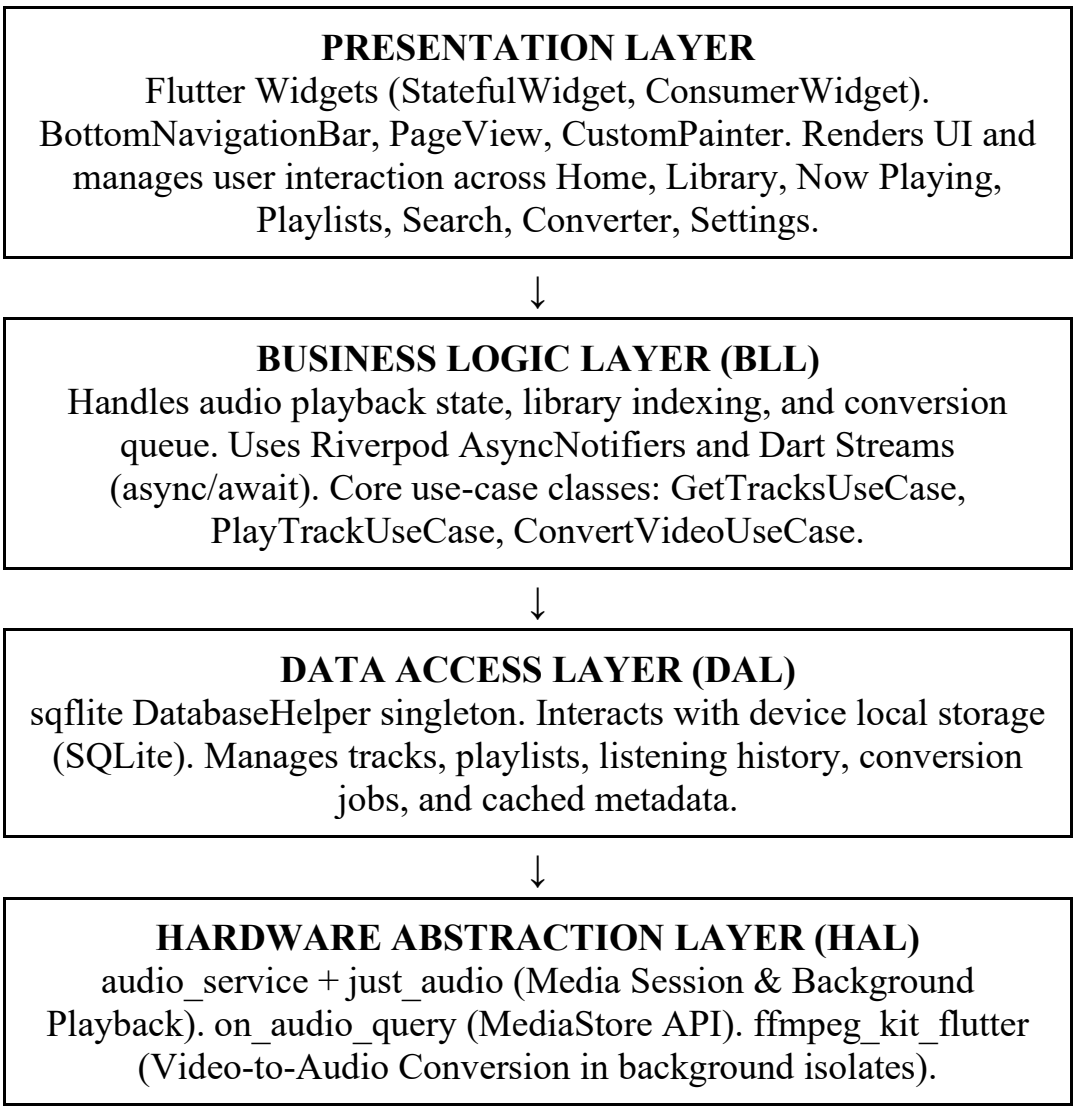
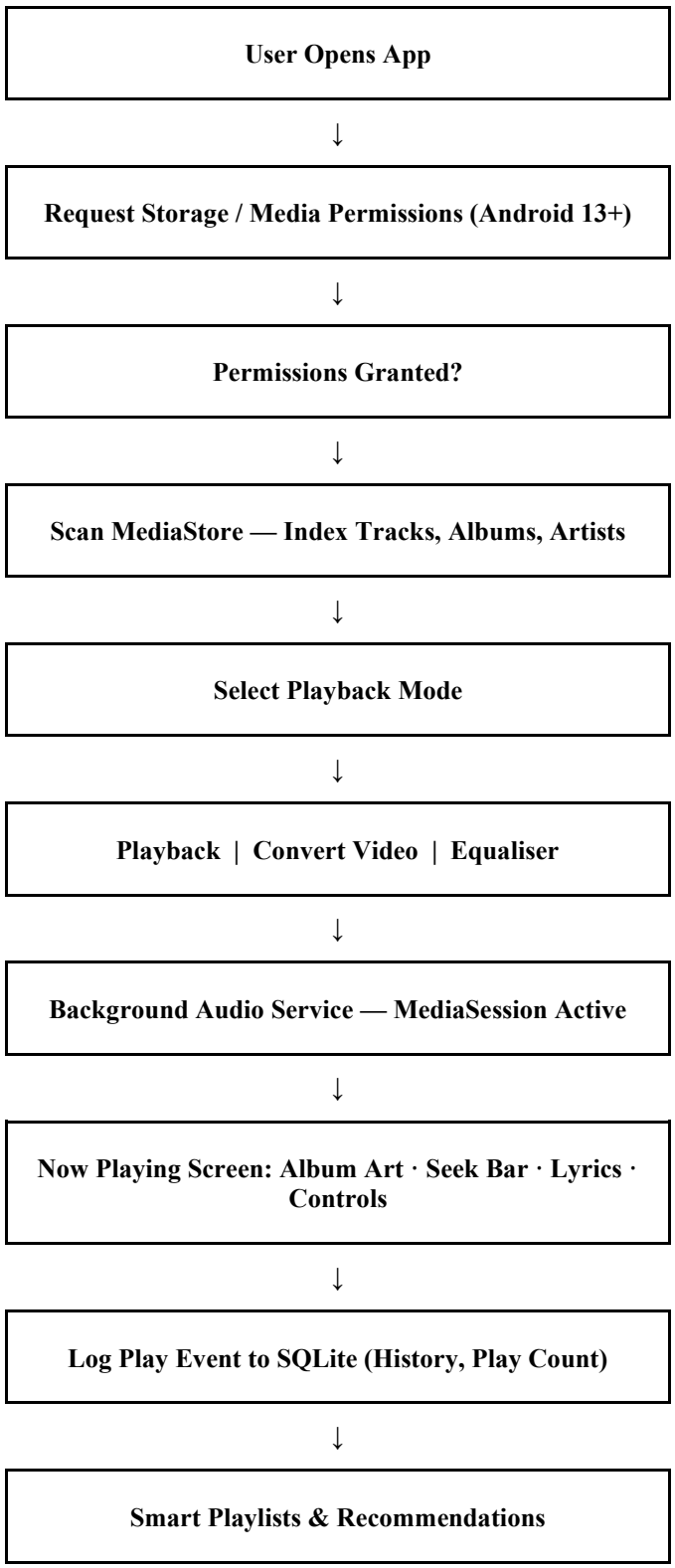


Fig 3.1 – Architecture Diagram

CHAPTER 4
FLOWCHART

The flowchart below illustrates the primary user journey through Jazz Music Player from application launch to active playback and smart playlist generation.



CHAPTER 5
CODE IMPLEMENTATION

5.1. Technology Overview

Language: Dart 3.x (compiled AOT for production).

Framework: Flutter SDK (Android SDK 21+).

State Management: Riverpod 2.x (AsyncNotifier, StateNotifier).

Persistence: sqflite (SQLite for Flutter).

Audio Engine: just_audio + audio_service.

Media Scanning: on_audio_query (MediaStore API).

Video Conversion: ffmpeg_kit_flutter (FFmpeg wrapper, background isolates).

Navigation: GoRouter with deep-link support.

Category	Libraries Used
Audio Playback	just_audio, audio_service, audio_session
Media Conversion	ffmpeg_kit_flutter_new_audio
Metadata & Scanning	audiotags, on_audio_query
Local Storage	sqflite, shared_preferences
File Handling	file_picker, path_provider, path
State Management	flutter_riverpod, riverpod_annotation
Navigation	go_router
Charts & Graphics	fl_chart, palette_generator
Utilities	uuid, cupertino_icons

5.2. Core Logic Implementation

The logic is driven by reactive programming using Dart Streams and Riverpod providers. A critical component is the AudioPlayerNotifier, which uses

audio_service to maintain an active media session across the system. This ensures that if the user is in the middle of a listening session or interacting with another app, playback context is never lost. Heavy tasks such as library indexing and FFmpeg conversion are offloaded to background Dart isolates to keep the main thread free for 60 fps rendering.

5.3. Detailed Code Implementation

Application Entry Point (main.dart)

```
// main.dart
import 'package:audio_service/audio_service.dart';
import 'package:flutter/material.dart';
import
'package:flutter_riverpod/flutter_riverpod.dart';
import 'core/services/music_handler.dart';
import 'core/theme/app_theme.dart';
import 'providers/app_providers.dart';
import 'routing/app_router.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final musicHandler = await AudioService.init(
    builder: () => MusicHandler(),
    config: const AudioServiceConfig(
      androidNotificationChannelId:
'com.jazzmusic.channel.audio',
      androidNotificationChannelName: 'Jazz Music
Playback',
      androidNotificationOngoing: true,
      androidStopForegroundOnPause: true,
    ),
  );
  runApp(
    ProviderScope(
      overrides: [

musicHandlerProvider.overrideWithValue(musicHandler),
      ],
      child: const JazzMusicApp(),
    ),
  );
}

class JazzMusicApp extends ConsumerWidget {
```

```
  const JazzMusicApp({super.key});
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final router = ref.watch(appRouterProvider);
    return MaterialApp.router(
      title: 'Jazz Music Player',
      debugShowCheckedModeBanner: false,
      theme: AppTheme.darkTheme(),
      routerConfig: router,
    );
  }
}
```

Database Initialisation (sqflite)

```
// database_helper.dart
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

class DatabaseHelper {
  static final DatabaseHelper _instance =
DatabaseHelper._internal();
  factory DatabaseHelper() => _instance;
  DatabaseHelper._internal();
  static Database? _database;

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDB();
    return _database!;
  }

  Future<Database> _initDB() async {
    final path = join(await getDatabasesPath(),
'jazz_music.db');
    return await openDatabase(path, version: 2,
      onCreate: _onCreate, onUpgrade: _onUpgrade);
  }

  Future<void> _onCreate(Database db, int version)
async {
    await db.execute('''
      CREATE TABLE tracks (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
        title TEXT NOT NULL, artist TEXT, album TEXT,
        duration INTEGER, filePath TEXT UNIQUE,
        albumArtPath TEXT, playCount INTEGER DEFAULT
0,
        lastPlayed INTEGER)
    ''');
    await db.execute('''
      CREATE TABLE playlists (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL, createdAt INTEGER)
    ''');
  }
}
```

Audio Player Notifier (Riverpod + just_audio)

```
// audio_player_notifier.dart
import 'package:just_audio/just_audio.dart';
import
'package:riverpod_annotation/riverpod_annotation.dart'
;

@riverpod
class AudioPlayerNotifier extends
_$AudioPlayerNotifier {
  late final AudioPlayer _player;

  @override
  AudioPlayerState build() {
    _player = AudioPlayer();
    _listenToPlayerState();
    return AudioPlayerState.initial();
  }

  void _listenToPlayerState() {
    _player.playerStateStream.listen((ps) {
      state = state.copyWith(isPlaying: ps.playing,
        processingState: ps.processingState);
    });
    _player.positionStream.listen((pos) {
      state = state.copyWith(position: pos);
    });
  }
```

```
  Future<void> playTrack(Track track) async {
    await
_player.setAudioSource(AudioSource.file(track.filePath
));
    await _player.play();
    state = state.copyWith(currentTrack: track);
  }

  Future<void> togglePlayPause() async =>
    _player.playing ? await _player.pause() : await
_player.play();

  Future<void> seek(Duration pos) =>
_player.seek(pos);
  Future<void> next()     => _player.seekToNext();
  Future<void> previous() => _player.seekToPrevious();
}
```

CHAPTER 6
CONCLUSION

Jazz Music Player represents a significant shift from generic audio applications to a specialised, performance-oriented ecosystem designed for the modern Android user. By moving beyond a "one-size-fits-all" approach, the application successfully addresses the unique audio demands of different listening environments and individual preferences such as lossless FLAC audiophile playback, portable MP3 streaming, and on-device video-to-audio conversion.

The integration of real-time background audio session management ensures that playback guidance and media controls remain dynamic, reflecting the user's actual listening activity rather than static estimates. Supported by a robust multi-layered Clean Architecture — leveraging the Flutter framework for high-performance rendering and a secure sqflite local data layer — Jazz Music Player provides a seamless, data-driven music experience.

Ultimately, this project bridges the gap between professional-grade audio tooling and everyday mobile usage, empowering users with the precision tools necessary to curate, discover, and enjoy their personal music collection on any Android device.

CHAPTER 7
FUTURE WORK

- Predictive Listening Analytics: Leveraging historical playback and user-behaviour data to predict "peak listening" windows and pre-buffer tracks before the user initiates playback.
- Deep Audio Fingerprinting: Using the AcoustID / Chromaprint library to automatically identify untagged tracks and fetch correct metadata from MusicBrainz.
- LLM-Powered Liner Notes: Integrating a lightweight Large Language Model to generate contextual artist bios, album histories, and mood-based descriptions within the Now Playing screen.
- Cross-Platform Expansion: Extending the Flutter codebase to iOS and desktop (macOS, Windows) with platform-specific audio engine adaptors while retaining shared domain and data layers.
- Cloud Sync for Playlists: Firebase Firestore integration for real-time playlist synchronisation across devices with conflict-resolution strategies for offline edits.
- Voice Command Support: Google Assistant integration enabling hands-free commands such as "Play my Jazz playlist" or "Skip to the next song".
- Audio Visualiser: Real-time FFT spectrum analyser rendered via Flutter's CustomPainter with GPU-accelerated compositing for a visually immersive Now Playing experience.
- Social and Competitive Modules: Introducing "Listening Communities" where users can share curated playlists, compete in music-trivia challenges, and discover tracks through peer recommendations.